



CSGO Round Winner Prediction

CSS 324 Final Project

Team Members

- Bauyrzhan Ilyas
- Muratov Alikhan
- Mendegaliyev Assylkhan
- Nurlan Arys
- Parpi Beybarys

1.TOPIC & MOTIVATION

Problem Statement

Counter-Strike: Global Offensive (CS:GO) is one of the world's most popular tactical FPS e-sports titles. Each round is a 2-team contest (Counter-Terrorists vs Terrorists) lasting at most ~175 seconds. The mid-round game state – health, economy, weapons, players alive – is highly informative about which team will ultimately win.

Goal: Build a binary classifier that predicts the round winner (CT or T) from a single mid-round snapshot.

Why It Matters

- E-sports analytics: Coaches use win-probability curves for tactical review (analogous to NFL win-probability models).
- Live broadcast overlays: Real-time win-probability enriches viewer experience on Twitch / YouTube.
- Game balance: Valve can detect map/economy imbalances across millions of matchmaking rounds.

Key Motivation Points

The ability to predict round outcomes from mid-round states has direct applications in professional esports broadcasting and coaching. Just as American football analytics have revolutionized how coaches make fourth-down decisions, CS:GO win-probability models can inform tactical timeout decisions, economy management, and round-strategy adjustments. Furthermore, game developers like Valve can use such models across millions of matchmaking rounds to identify systemic imbalances in maps or weapon economies that might otherwise only surface through professional play.

2. DATASET & PREPARATION

Data Source

- Source: Kaggle – CS:GO Round Snapshots
- Raw rows: 122,410
- Raw columns: 97

Data Loading

```
df_raw = pd.read_csv("csgo_round_snapshots.csv")
```

Final Clean Dataset

- Clean shape: 117,448 rows × 97 columns
- Duplicates removed: 4,962 rows ($\approx 4.05\%$ of raw data)

Feature Engineering

Will be prepared as separate paper blank

Rifle Columns (CT and T)

```
ct_rifle_cols = [  
    "ct_weapon_ak47", "ct_weapon_m4a4", "ct_weapon_m4a1s",  
    "ct_weapon_aug", "ct_weapon_famas", "ct_weapon_galilar", "ct_weapon_sg553"  
]
```

```
t_rifle_cols = [  
    "t_weapon_ak47", "t_weapon_m4a4", "t_weapon_m4a1s",  
    "t_weapon_aug", "t_weapon_famas", "t_weapon_galilar", "t_weapon_sg553"  
]
```

Target Encoding

```
df["winner_encoded"] = (df["round_winner"] == "T").astype(int)
```

Final Feature Count

- Total features after engineering: 108 columns
- Original raw features used as-is: CT/T scores, health, armor, money, player counts, weapon counts, grenade counts, helmets, defuse kits
- Engineered features added: 8 advantage features + 2 aggregated rifle counts + 2 aggregated grenade counts

3. EXPLORATORY DATA ANALYSIS (EDA)

Target Distribution Analysis

Round Winner Distribution:

- T: 59,941 (51.0%)
- CT: 57,507 (49.0%)

EDA Visualizations Produce

ALL VISUALIZATIONS WILL BE SHOWED IN THE NEXT PAPER

Key EDA Insights (Summarized)

1. Dataset is well-balanced (T 51% / CT 49%) – no resampling needed.
2. Player count advantage is the strongest single predictor – intuitive, as numbers matter tremendously in CS:GO.
3. Bomb planted shifts T win probability up by ~20 percentage points – reflects the fundamental asymmetry of the bomb defusal game mode.
4. Health & money advantages clearly separate CT/T win distributions – visible in both histograms and box plots.
5. Map effects exist but are modest – de_nuke and de_overpass tilt CT-side by ~5-7%; de_dust2 is near 50/50; de_vertigo shows anomalous patterns.
6. No class imbalance problem – stratification preserved during train/test split without needing SMOTE or other resampling.

EDA Part is over on this paper

4. MACHINE LEARNING EXPERIMENTS

Train/Test Split Configuration

Split Statistics:

- Total features: 108
- Train samples: 93,958
- Test samples: 23,490
- Class balance maintained: CT 49.0% / T 51.0% (via stratify=y)

Scaling Strategy

Scaling only applied to Logistic Regression (which requires scaled features for coefficient interpretability and convergence). Random Forest and Gradient Boosting handle unscaled features natively as they are tree-based algorithms.

Logistic Regression

Metric	Value
Accuracy	0.7576
AUC-ROC	0.8554
F1 Score (weighted)	0.7577

Random Forest (100 trees)

Metric	Value
Accuracy	0.8358
AUC-ROC	0.9171
F1 Score (weighted)	0.8358
Training Time	3.0 seconds

Random Forest significantly outperforms Logistic Regression (+7.8% accuracy, +6.2% AUC). The AUC of 0.917 indicates excellent class separation.

Gradient Boosting (100 trees, depth 4)

Metric	Value
Accuracy	0.7731
AUC-ROC	0.8748
F1 Score (weighted)	0.7729
Training Time	31.2 seconds

Surprisingly, default-parameter Gradient Boosting underperformed Random Forest (77.3% vs 83.6% accuracy) while taking ~10× longer to train. This suggests the default hyperparameters (especially `learning_rate=0.1` and `max_depth=4`) may not be optimal for this dataset, or that RF's bagging approach better handles the feature space.

Hyperparameter Tuning – Random Forest

Interpretation of Best Params:

- `max_depth: null` – Trees are allowed to grow fully without depth restriction, capturing complex interactions
- `min_samples_leaf: 3` – Relatively small leaf size allows fine-grained decisions
- `n_estimators: 200` – More trees improve ensemble stability and accuracy

Cross-Validation Results:

- Best CV accuracy: 0.8241
- Test accuracy after tuning: 0.8495

Improvement: +1.37% accuracy over untuned Random Forest

Performance Comparison Summary

Model	Accuracy	AUC-ROC	F1 Score	Training Time
Logistic Regression	0.7576	0.8554	0.7577	1.4s
Random Forest (100 trees)	0.8358	0.9171	0.8358	3.0s
Gradient Boosting (default)	0.7731	0.8748	0.7729	31.2s
RF (Tuned)	0.8495	0.9279	0.8495	~24 fits

Classification Reports (Detailed)

ROC Curves Analysis

The ROC curves show:

- Logistic Regression: AUC = 0.855 – decent discrimination
- Random Forest (untuned): AUC = 0.917 – strong discrimination
- Gradient Boosting: AUC = 0.875 – good but less than RF
- RF (Tuned): AUC = 0.928 – excellent discrimination

The tuned RF achieves nearly ideal ROC characteristics: high true positive rate (~0.85 at false positive rate ~0.15).

Confusion Matrices

- True Negatives (CT correct): 9,891
- False Positives (CT misclassified as T): 1,611
- False Negatives (T misclassified as CT): 1,882
- True Positives (T correct): 10,106

Accuracy calculation: $(9,891 + 10,106) / 23,490 = 20,000 / 23,490 \approx 0.851$

Feature Importance Analysis

Rank	Feature	Importance	Type
1	armor_advantage	~0.115	Advantage
7	health_advantage	~0.045	Advantage
8	t_armor	~0.040	T feature
9	players_advantage	~0.035	Advantage
10	money_advantage	~0.020	Advantage
11	grenade_advantage	~0.015	Advantage
12	ct_money	~0.010	CT feature
13	t_money	~0.005	T feature
4	ct_armor	~0.055	CT feature
5	rifle_advantage	~0.045	Advantage
6	ct_total_rifles	~0.100	CT feature

Key Observations:

- armor_advantage is the single most important feature – team armor differences strongly predict outcomes
- t_armor appears as a distinct important feature, indicating T team armor is particularly predictive
- Advantage features dominate the top ranks, confirming the engineering strategy was sound
- late_round appears in top 20 but with lower importance than economy/health features

Model Comparison Visualizations

Comparisons showed in paper blanks

5. FINAL MODEL & DEMO DESCRIPTION

Final Model Selection: Random Forest (Tuned)

Why Random Forest (Tuned) was selected as the final model:

Criterion

Evaluation

1. Highest test accuracy and AUC-ROC

0.8495 accuracy, 0.9279 AUC — best among all models evaluated

2. Handles mixed numeric/integer features natively

No scaling required for inference, simplifying deployment

3. Feature importances give interpretable insights

Domain-consistent: armor_advantage #1, confirms CS:GO intuition

4. Fast inference (~1 ms/prediction)

Suitable for real-time broadcast overlays (30+ predictions per second)

5. Robust to outliers

Tree-based methods are inherently robust to extreme values without preprocessing

6.Interactive Demo Function

Function Logic

Create feature vector:

Populate a pandas Series with 108 feature columns (all zeros initially)

Fill in provided parameters for core features

Compute engineered features (health_advantage, money_advantage, etc.)

Encode map name via pre-fitted LabelEncoder (`le_map`)

Make prediction:

```
prob = best_rf.predict_proba(pd.DataFrame([row[feature_cols]]))[0]
```

```
pred = best_rf.predict(pd.DataFrame([row[feature_cols]]))[0]
```

```
winner = "T" if pred == 1 else "CT"
```

```
ct_pct, t_pct = prob[0]*100, prob[1]*100
```

Display results (if verbose=True):

+-----+

ROUND PREDICTION

CT Win Prob : 46.0% 

T Win Prob : 54.0% 

-> Predicted Winner : T

+-----+

Demo Scenarios

Five pre-defined scenarios demonstrate model behavior across different game states.

Scenario 1: 5v5 Pistol Round – Game Start

Input parameters:

time_left=170, ct_health=500, t_health=500,

ct_money=800, t_money=800,

ct_players_alive=5, t_players_alive=5,

ct_awp=0, t_awp=0, ct_rifles=0, t_rifles=0

Result:

- CT Win Prob: 46.0%
- T Win Prob: 54.0%
- Predicted Winner: T

Scenario 2: CT Eco Round – T Full Buy with AWP

Input parameters:

time_left=100, ct_health=400, t_health=500,
ct_money=1000, t_money=20000,
ct_players_alive=5, t_players_alive=5,
ct_awp=0, t_awp=2, ct_rifles=0, t_rifles=4

Result:

- CT Win Prob: 31.6%
- T Win Prob: 68.4%
- Predicted Winner: T

Scenario 3: 4 CTs vs 1 T – Bomb Planted, 15s Left

Input parameters:

time_left=15, ct_health=360, t_health=20,
ct_money=5000, t_money=1000,
ct_players_alive=4, t_players_alive=1,
ct_awp=1, t_awp=0, bomb_planted=True

Result:

- CT Win Prob: 78.4%
- T Win Prob: 21.6%
- Predicted Winner: CT

Scenario 4: 1v1 – CT AWP vs T AK47

Input parameters:

time_left=60, ct_health=100, t_health=100,
ct_money=4000, t_money=4000,
ct_players_alive=1, t_players_alive=1,
ct_awp=1, t_awp=0, ct_rifles=0, t_rifles=1

Result:

- CT Win Prob: 54.1%
- T Win Prob: 45.9%
- Predicted Winner: CT

Scenario 5: 3v3 Even Economy – Bomb Planted, 25s Left

Input parameters:

time_left=25, ct_health=300, t_health=300,
ct_money=6000, t_money=6000,
ct_players_alive=3, t_players_alive=3,
ct_awp=0, t_awp=0, bomb_planted=True

Result:

- CT Win Prob: 32.3%
- T Win Prob: 67.7%
- Predicted Winner: T

6. ANALYSIS & CONCLUSIONS

Model Strengths

1. High discriminative power — AUC of 0.928 indicates excellent separation of CT and T rounds
2. Well-calibrated probabilities — Predicted win probabilities closely match actual win rates (Figure: win_prob_over_time)
3. Interpretable — Feature importances align with CS:GO domain knowledge
4. Fast inference — Suitable for real-time applications (~1ms per prediction)
5. No class bias — Balanced precision/recall for both CT and T
6. Robust preprocessing — Sanity clamping prevents outlier-induced errors

